

编译原理第二章词法分析——考试复习总结

中南大学计算机学院

2026 年 5 月 12 日

目录

1	词法分析简介	2
2	词法分析器的手工构造	3
3	正则表达式 (Regular Expression, RE)	4
4	有限状态自动机 (Finite Automaton, FA)	6
5	正则表达式转 NFA ——Thompson 算法	7
6	NFA 转 DFA ——子集构造算法 (Subset Construction)	8
7	DFA 的最小化——Hopcroft 算法	10
8	DFA 的代码表示与驱动	11
9	核心考点速记	12

1 词法分析简介

编译器的阶段结构

编译器通常分为前端（**Front End**）和后端（**Back End**）。前端负责将源代码转换为中间表示（IR），后端负责将 IR 转换为目标代码。[p.5]

前端内部的三个阶段依次为：[p.6]

- (1) 词法分析器（**Lexer/Scanner**）：字符流 → 记号流（Token Stream）
- (2) 语法分析器（**Parser**）：记号流 → 语法树（Syntax Tree）
- (3) 语义分析器（**Semantic Analyzer**）：语法树 → 中间表示（IR）

词法分析器的任务

词法分析器将输入的字符流（**Character Stream**）转换为记号流（**Token Stream**）。[p.7, p.9]

- 字符流：和被编译的语言密切相关（如 ASCII、Unicode 等）
- 记号流：编译器内部定义的数据结构，编码所识别出的词法单元

记号的数据结构定义

记号的典型数据结构（以 C 语言为例）[p.8]：

```
enum kind {IF, LPAREN, IDENT, INT, GT, ...};  
struct token {  
    enum kind k;    // 单词类别  
    char *lexeme;    // 单词自身的字符串值（关键字/界符可为NULL）  
};
```

例如，输入 `if (x>5)` 将被切分为 6 个记号（IF, LPAREN, IDENT, GT, INT, RPAREN）。

词法分析输出的单词分类

程序语言的基本语法符号（单词）一般分为以下五种 [p.14–16]：

- (1) 关键字（**Keyword**）：如 `if`, `while`, `else`。个数确定，可全体编为一类，也可一字一类。
- (2) 标识符（**Identifier**）：个数不确定，统一作为一类。自身值是标识符的字符串。
- (3) 常数（**Constant**）：各类型常数（整数、浮点数等），个数不确定，按类型分类。自身值是常数的二进制数值。
- (4) 运算符（**Operator**）：如 `+`, `-`, `*`, `/`, `<` 等。个数确定，通常一符一类。
- (5) 界符（**Delimiter**）：如 `,`, `;`, `(`, `)`, `:` 等。个数确定，通常一符一类。

词法分析程序输出的单词符号通常用二元式表示：[p.14–17]

(单词类别, 单词内部码值)

词法分析器的两种实现方案

[p.13, p.33]

- **手工编码实现法 (Hand-coded)**: 相对复杂且容易出错, 但是目前非常流行的实现方法 (如 GCC、LLVM 均采用手工编码)。
- **词法分析器的生成器 (Generator-based)**: 可快速原型、代码量较少, 但较难控制细节 (如 Lex、Flex、JLex 等)。

2 词法分析器的手工构造

状态转换图

状态转换图 (State Transition Diagram) 是一张有限方向图。[p.19]

- 用结点代表**状态**, 状态之间用箭弧 (有向边) 连接。
- 箭弧上的标记 (字符) 代表在射出结点状态下可能出现的输入字符或字符类。
- 包含有限个状态, 有一个**初态 (Start State)**, **终态 (Accepting State)** 用双圈表示。

典型例子: 标识符的状态转换图 [p.19]

- S (开始状态) $\xrightarrow{\text{字母}}$ I (标识符状态) $\xrightarrow{\text{字母/数字}}$ I, 遇到非字母/数字时进入 E (终态)

状态转换图的实现方法

每个结点对应一段程序, 前面状态结点的程序调用其后继结点的程序。[p.21]

实现所需的**标准过程与变量** [p.22]:

- **Char** (全程字符变量): 存放最新读入的源程序字符
- **TOKEN** (字符串): 存放构成单词符号的字符串
- **GETChar**: 读入一个源程序字符, 搜索指针前移
- **GETNBC**: 反复调用 GETChar, 直到读入非空格字符
- **CONCAT**: 把 Char 中字符连接到 TOKEN 末尾
- **Letter/Digit**: 判别 Char 中是否为字母/数字
- **Return(c, val)**: 返回单词类别和内部码值
- **Retract**: 搜索器指针回退一个字符

标识符和关键字的识别方法

在很多语言中（如 C 语言），关键字从词法角度来看是标识符的一部分。两种识别方法：[p.27]

方法一：切分转移图 (Split Transition Diagram) [p.28-29] 将关键字（如 if, while 等）在状态转换图中单独切分出分支。这种方法会导致状态图变得较复杂。

方法二：关键字表算法 (Keyword Table Algorithm) [p.30]

1. 对给定语言中所有的关键字，构造关键字构成的哈希表 H。
2. 对所有的标识符和关键字，先统一按标识符的转移图进行识别。
3. 识别完成后，进一步查表 H 看是否是关键字。
4. 通过合理构造哈希表（完美哈希），可以在 $O(1)$ 时间内完成查表。

3 正则表达式 (Regular Expression, RE)

基本概念

[p.34-36]

- 字母表 Σ ：元素的非空有限集合，如 $\Sigma = \{a, b, c\}$ 。
- 字符 (Symbol)：字母表 Σ 的一个元素。
- 字 (String)： Σ 上字符的有穷序列。
- 空字 ε ：不含任何字符的字，长度为 0。
- 字的长度： $|\alpha|$ （其中 $|\varepsilon| = 0$ ）。
- Σ^* ： Σ 上所有字的全体（包含 ε ）。
- 字的连接：字 α 与字 β 的连接记为 $\alpha\beta$ 。
- 字的方幂： $\alpha^n = \underbrace{\alpha\alpha\cdots\alpha}_{n\text{次}}$ ，特别地， $\alpha^0 = \varepsilon$ 。
- 字的集合 A 与 B 的乘积： $AB = \{\alpha\beta \mid \alpha \in A \wedge \beta \in B\}$ 。
- Σ^* 子集的方幂： $A^0 = \{\varepsilon\}$, $A^1 = A$, $A^n = A^{n-1}A$ 。
- 正则闭包 (Positive Closure)： $A^+ = A^1 \cup A^2 \cup \cdots$
- 闭包 (Kleene Closure)： $A^* = A^0 \cup A^+ = \{\varepsilon\} \cup A^+$

正规式 (Regular Expression) 与正规集 (Regular Set)

正规集是字母表 Σ 上的一个特殊字集，借助正规式来描述。定义是递归的 [p.37]：

1. ε 和 ϕ （空集）都是 Σ 上的正规式，所表示的正规集分别为 $\{\varepsilon\}$ 和 $\emptyset$ 。
2. 任何 $a \in \Sigma$, a 是 Σ 上的正规式，所表示的正规集为 $\{a\}$ 。

3. 假定 u 和 v 是正规式, 正规集分别为 $L(u)$ 和 $L(v)$, 则:

- $u|v$ 是正规式, 正规集为 $L(u) \cup L(v)$ (选择/或)
- uv 是正规式, 正规集为 $L(u)L(v)$ (连接)
- u^* 是正规式, 正规集为 $(L(u))^*$ (Kleene 闭包)

仅由有限次使用上述三步定义的表达式才是 Σ 上的正规式, 仅由这些正规式表示的字集才是 Σ 上的正规集。

优先级

[p.38] 正规式运算优先级递增为: $|$ (或) $<$ 连接 $(\cdot)$ $<$ $*$ (闭包)。对应正规集运算为: $\cup < \cap < *$ 。

正规式的性质

[p.39]

1. 交换律: $u|v = v|u$
2. 结合律: $(u|v)|w = u|(v|w)$, $(uv)w = u(vw)$
3. 分配律: $u(v|w) = uv|uw$, $(u|v)w = uw|vw$
4. 单位元: $\varepsilon u = u\varepsilon = u$

正则表达式的归纳定义 (另一种表述)

对给定字符集 $\Sigma = \{c_1, c_2, \dots, c_n\}$: [p.41]

$$e \rightarrow \varepsilon \mid c \in \Sigma \mid e|e \mid ee \mid e^*$$

典型例子

- C 语言关键字 if: 正则表达式为 `if` [p.43]
- C 语言标识符 (字母 | 下划线开头, 后跟零或多个字母、数字、下划线): [p.44]

$$(a|b|\dots|z|A|B|\dots|Z|_)(a|b|\dots|z|A|B|\dots|Z|0|1|\dots|9|_)^*$$

- C 语言无符号十进制整数 (0 或 1-9 开头后跟 0-9): [p.45]

$$0 \mid ((1|2|\dots|9)(0|1|2|\dots|9)^*)$$

语法糖 (Syntactic Sugar)

为简化构造, 引入辅助表示: [p.46]

- $[c_1-c_n]$: 等价于 $c_1|c_2|\dots|c_n$ (字符类)
- e^+ : 等价于 e^*e , 一个或多个 e (正闭包)

- $e?$: 等价于 $e|\varepsilon$, 零个或一个 e (可选)
- $e\{i,j\}$: i 到 j 个 e 的连接 (重复次数)
- ``. '`: 除换行符 \n 外的任意字符`

4 有限状态自动机 (Finite Automaton, FA)

自动机的五元组定义

一个有限状态自动机 M 定义为五元组: [p.50-51]

$$M = (\Sigma, S, q_0, F, \delta)$$

其中:

- Σ : 字母表 (有限非空符号集)
- S : 状态集 (有限非空)
- $q_0 \in S$: 初始状态
- $F \subseteq S$: 终结状态集 (接受状态集)
- δ : 转移函数

DFA vs NFA

[p.52]

- 确定有限自动机 (DFA, Deterministic FA): 对任意的字符, 最多有一个状态可以转移。

$$\delta : S \times \Sigma \rightarrow S$$

- 非确定有限自动机 (NFA, Nondeterministic FA): 对任意的字符, 有多于一个状态可以转移。

$$\delta : S \times (\Sigma \cup \{\varepsilon\}) \rightarrow \mathcal{P}(S)$$

($\mathcal{P}(S)$ 表示 S 的幂集)

关键区别:

- (1) DFA 的转移函数是单值的; NFA 可以是多值的。
- (2) NFA 允许 ε 转移 (不消耗任何输入字符的转移); DFA 不允许。
- (3) DFA 便于直接编码实现 (用转移表即可); NFA 执行时需要大量遍历和回溯。

DFA 的实现：转移表

给定 DFA 的转移表（Transition Table），以一个二维数组存储状态转换：[p.53]

状态\字符	a	b
0	1	0
1	2	1
2	2	2

词法分析驱动代码通过查表驱动状态转移，简单高效。

自动生成词法分析器的完整流程

[p.55, p.60, p.86, p.97]

正则表达式（RE） $\xrightarrow{\text{Thompson 算法}}$ NFA $\xrightarrow{\text{子集构造算法}}$ DFA $\xrightarrow{\text{Hopcroft 最小化}}$ 最小 DFA $\xrightarrow{\text{代码生成}}$ 词法分析器代码

5 正则表达式转 NFA —— Thompson 算法

算法概览

Thompson 算法基于对正则表达式结构的归纳：[p.56]

- 对基本的正则表达式（ ε 和单个字符）直接构造 NFA
- 对复合的正则表达式（选择、连接、闭包）递归构造 NFA
- 递归算法，容易实现

基本构造规则（Thompson 构造法）

[p.57]

(1) 空串 ε :

$$i \xrightarrow{\varepsilon} f$$

新增两个状态 i 和 f ，从 i 到 f 有一条 ε 转移边。

(2) 单个字符 $c \in \Sigma$:

$$i \xrightarrow{c} f$$

新增两个状态 i 和 f ，从 i 到 f 有一条标号为 c 的转移边。

(3) 连接 $e_1 e_2$:

NFA(e_1) 的终态与 NFA(e_2) 的初态合并（或通过 ε 转移连接）。

(4) 选择 $e_1 | e_2$:

新增初态 i 和终态 f , i 通过两条 ε 边分别指向 $\text{NFA}(e_1)$ 和 $\text{NFA}(e_2)$ 的初态; 两个 NFA 的终态分别通过 ε 边指向 f 。

(5) 闭包 e_1^* :

新增初态 i 和终态 f , i 通过 ε 指向 $\text{NFA}(e_1)$ 的初态和 f ; $\text{NFA}(e_1)$ 的终态通过 ε 指向 f 和 $\text{NFA}(e_1)$ 的初态 (形成回路)。

示例

对正则表达式 $a(b|c)^*$, 使用 Thompson 算法构造 NFA, 得到 9 个状态的 NFA [p.58]。该 NFA 包含多个 ε 转移和非确定性转移, 需要通过子集构造算法进一步转化为 DFA。

拓展: 正则表达式转 NFA 的代码实现

[p.104–108] 可以通过以下步骤实现:

1. **中缀转后缀**: 使用运算符栈, 将正则表达式从中缀形式转换为后缀形式 (后缀表达式中运算符位于操作数之后, 无需括号体现优先级)。
2. **后缀生成语法树 (REG 节点树)**: 使用栈存储中间结果, 遇到运算符时出栈生成 REG 节点再入栈。
3. **对语法树递归生成 NFA**。

REG 节点数据结构如下: [p.104]

```
enum kind {eps, c, and, or, closure};
struct reg {
    enum kind k;
    reg *left;
    reg *right;
    char *c;
};
```

6 NFA 转 DFA ——子集构造算法 (Subset Construction)

为什么 NFA 要转 DFA?

[p.61] NFA 在执行时, 对于每个输入字符, 可能需要查看所有可能的状态转移结果, 需要大量遍历和回溯, 效率极低。DFA 对每个输入字符最多只有一个确定的状态转移, 可以直接用转移表驱动, 效率高。

-闭包 (ε -Closure)

[p.71] 设 I 是 NFA M' 的状态集的一个子集, 定义 $\varepsilon\text{-CLOSURE}(I)$ 为:

- (1) 若 $s \in I$, 则 $s \in \varepsilon\text{-CLOSURE}(I)$

(2) 若 $s \in I$, 则从 s 出发经过任意条 ε 弧而达到的状态 s' , 有 $s' \in \varepsilon\text{-CLOSURE}(I)$

例: 若 $I = \{1, 3\}$, 则 $\varepsilon\text{-CLOSURE}(I) = \{1, 3, 2, 8\}$ (视具体 NFA 图而定)。[p.71]

I_a 的定义

[p.72] 假定 I 是 M' 的状态子集, a 是 Σ 中的字符, 定义:

$$I_a = \varepsilon\text{-CLOSURE}(J)$$

其中 J 是 I 中任意状态出发 (跳过前面任意多条 ε 弧), 经过一条 a 弧而能达到的状态结点全体。

子集构造算法步骤

[p.73–74, p.79–80] 设 $\Sigma = \{a_1, a_2, \dots, a_m\}$:

1. 构造表格: 第 0 列为状态子集 I , 第 1 至 m 列分别为 $I_{a_1}, I_{a_2}, \dots, I_{a_m}$ 。
2. 第 1 行第 0 列设为 $\varepsilon\text{-CLOSURE}(\{q_0\})$ (初始状态的 闭包)。
3. 依次计算当前行每个 I_{a_i} , 如果 I_{a_i} 未曾在第 0 列中出现过, 则将其作为新的一行插入第 0 列。
4. 重复步骤 3, 直至所有 I_{a_i} 均已在第 0 列中出现过为止。
5. 每一行 (即每一个状态子集) 作为新 DFA 的一个状态。含有原 NFA 终态的子集为 DFA 的新终态。

终止性: 因为 S 是有限状态集, S 中状态的组合数也是有限的, 即 $|S^*| \leq 2^{|S|}$, 算法必然在有限步内终止。[p.73]

时间复杂度: 最坏情况为 $O(2^N)$ (N 为 NFA 状态数), 但在实际中不常发生, 因为并不是每个子集都会出现。[p.81]

-闭包的计算

两种计算方法: [p.82–83]

- **深度优先 (DFS):** 从初始状态出发, 沿着 ε 边递归深入。
- **宽度优先 (BFS):** 从初始状态出发, 逐层访问所有 ε 可达状态。

完整示例: 识别二进制偶数的 DFA

以正规式 $1(1|0)^*0$ 为例 (不以 0 开头的无符号二进制偶数): [p.75–76]

1. 写出正规式: $1(1|0)^*0$
2. 构造 NFA
3. 使用子集构造算法确定化为 DFA (得到 5 行状态的转移表)

7 DFA 的最小化——Hopcroft 算法

目的

寻找一个状态数比原 DFA M 少的 DFA M' ，使得 $L(M) = L(M')$ 。[p.90]

基本概念

[p.90–91]

- **等价状态 (Equivalent States):** DFA 的两个状态 s 和 t ，如果从 s 出发能读出某个字 w 而停于终态，当且仅当从 t 出发也能读出同样的字 w 而停于终态。
- **可区别状态 (Distinguishable States):** 不等价的 DFA 状态。如果存在某个字符串 w 使状态 s 和 t 一个能到达终态而另一个不能，则称 w 区别了 s 和 t 。
- **等价状态:** 无法被任何字符串区别开的两个状态。
- **死状态 (Dead State):** 无法到达终止状态的非终止状态。
- **不可达状态 (Unreachable State):** 不能从开始状态到达的状态。

Hopcroft 算法步骤

[p.91–93] 基本思想：把状态集划分成互不相交的子集，使子集中的状态互相等价。

1. **初始划分:** 将 S 分为两个子集——终态集 A 和非终态集 N 。显然，属于不同子集的状态是可区别的。
2. **迭代细分:** 假设当前划分 $\Pi = \{I^{(1)}, I^{(2)}, \dots, I^{(m)}\}$ ，对于每个子集 $I^{(i)}$:
 - 设 $I^{(i)} = \{q_1, q_2, \dots, q_k\}$
 - 若存在某个输入字符 a ，使得 $I_a^{(i)}$ (该子集中状态经 a 转移后到达的新状态集合) 不全包含在当前划分 Π 的某一个子集中，则将 $I^{(i)}$ 一分为二 (或分为更多份)。
 - 具体地，若 $I_a^{(i)}$ 落入 Π 中 N 个不同子集，则将 $I^{(i)}$ 划分为 N 个不相交的组，使得每个组经 a 转移后都落入 Π 的同一个子集中。
3. **重复:** 重复步骤 2，直至划分中所含的子集数不再增长为止。此时每个子集中的状态互相等价，不同子集中的状态互相可区别。
4. **合并:** 取每组状态中的代表状态，删去其他等价状态。将导入被删状态的弧都改指向代表状态。
5. **删除无用状态:** 删除死状态和不可达状态。

具体划分示例 [p.94]

- 初始划分 $\Pi = \{N, A\}$ (非终态组 + 终态组)
- 对 N 组检查是否可继续划分：如果 N 中的某状态和另一状态经某个字符转移后落入不同组，则拆分

- 重复至 Π 不再变化

另一个示例 [p.95]: 5 个状态 $\{q_0, q_1, q_2, q_3, q_4, q_5\}$ 通过 e 和 i 字符进行划分, 初始 $\{N, A\}$, 经 e 划分得到 $\{\{q_0, q_1\}, \{q_2, q_4\}, \{q_3, q_5\}\}$, 再经 e/i 进一步细分。

正规式与有限自动机的等价性

两个重要结论: [p.64–70]

1. Σ 上 NFA M 所能识别的字的全体是 Σ 上的一个正规集。
 - 证明方法: 对 NFA 消结, 直至剩初态和终态, 弧上标记为一个正规式。
2. 对于 Σ 上的每个正规集 V , 存在一个 Σ 上的 DFA M , 使得 $V = L(M)$ 。
 - 证明方法: 先用正规式构造 NFA M' (逐步分解法), 再将 NFA 确定化为 DFA。

推论: 一个字集是正规的, 当且仅当它可由一个 DFA (或 NFA) 所识别。[p.74]

消结规则 (NFA 化简为正规式): [p.65–67] 设要消除的状态结点 k , 其入边标号为 R_1 (来自状态 i), 出边标号为 R_2 (去往状态 j), 自身回路标号为 R_3 。消除后, 从 i 到 j 的新边标号为 $R_1(R_3)^*R_2$ (与原有边的标号取并)。

8 DFA 的代码表示与驱动

三种表示方式

[p.98]

- **转移表 (Transition Table):** 用二维数组存储, 行列分别表示状态和输入字符。
- **跳转表 (Jump Table):** 类似于汇编中的跳转表, 以状态为索引跳转到对应的代码段。
- **哈希表 (Hash Table):** 以 (状态, 字符) 二元组为键, 目标状态为值。

转移表驱动的词法分析器架构

[p.99] 转移表 + 驱动代码 $\rightarrow$ 词法分析器:

- **转移表:** 存储 DFA 的状态转移关系
- **驱动代码:** 读取字符、查表、状态转移、识别 Token

驱动代码中的最长匹配策略

[p.101] 在词法分析中, 当存在多个可能匹配的模式时采用**最长匹配 (Longest Match)**原则:

- 例如: 关键字 `if` 和 `ifif`, 输入为 `ifif` 时匹配 `ifif` 而非 `if`
- 驱动代码维护当前 Accept 状态和出错状态, 在无法继续匹配时回退到最近的 Accept 状态

拓展：NFA 的 FPGA 加速

[p.84] 利用 FPGA（现场可编程逻辑门阵列）可以定制并行电路逻辑，同时执行状态机的多个状态转移分支。对于 NFA 这种在 CPU 上串行执行效率极低的结构，FPGA 可以通过并行计算大幅提升匹配速度。

9 核心考点速记

关键算法步骤速查

1. Thompson 算法 (RE $\rightarrow$ NFA)

- 对 ε : $i \xrightarrow{\varepsilon} f$
- 对 $c \in \Sigma$: $i \xrightarrow{c} f$
- 对 $e_1|e_2$: 新初态 $\xrightarrow{\varepsilon}$ NFA(e_1) 初态与 NFA(e_2) 初态; 两 NFA 终态 $\xrightarrow{\varepsilon}$ 新终态
- 对 e_1e_2 : NFA(e_1) 终态 $\xrightarrow{\varepsilon}$ NFA(e_2) 初态
- 对 e_1^* : 新初态 $\xrightarrow{\varepsilon}$ NFA(e_1) 初态和新终态; NFA(e_1) 终态 $\xrightarrow{\varepsilon}$ NFA(e_1) 初态和新终态

2. 子集构造算法 (NFA $\rightarrow$ DFA)

1. 计算初始状态的 ε -闭包, 作为 DFA 初态
2. 对每个新 DFA 状态 (NFA 状态子集), 计算每个输入字符下的转移目标 ($I_a = \varepsilon$ -闭包 (J))
3. 新产生的子集作为新 DFA 状态加入表中
4. 重复至无新状态产生

3. Hopcroft 最小化算法 (DFA $\rightarrow$ 最小 DFA)

1. 初始划分 $\Pi = \{\text{非终态集}, \text{终态集}\}$
2. 对每个组检查是否可被某输入字符细分
3. 重复至 Π 不再变化
4. 合并等价状态, 删除无用状态

关键公式速查

概念	公式/定义
ε -CLOSURE(I)	$\{s \mid s \in I\} \cup \{s' \mid s \in I, s \text{ 经 } \varepsilon \text{ 可达 } s'\}$
I_a (子集构造)	$\varepsilon\text{-CLOSURE}(\{t \mid s \in I, s \xrightarrow{a} t\})$
正则闭包	$A^+ = A^1 \cup A^2 \cup \dots$
Kleene 闭包	$A^* = A^0 \cup A^+ = \{\varepsilon\} \cup A^+$
A^* (等价式)	$A^* = \varepsilon \mid A^+$
DFA 转移函数	$\delta : S \times \Sigma \rightarrow S$
NFA 转移函数	$\delta : S \times (\Sigma \cup \{\varepsilon\}) \rightarrow \mathcal{P}(S)$
子集构造最坏复杂度	$O(2^{ S })_{\text{NFA 状态数}}$

概念对照表

对比维度	DFA	NFA
转移确定性	确定的（每字符最多一转移）	非确定的（可多转移）
ε 转移	不允许	允许
转移函数值域	S (单个状态)	$\mathcal{P}(S)$ (状态集合)
实现难度	直接可用转移表	需回溯/遍历
状态数	指数级（相对 NFA）	较少
执行效率	高（ $O(\text{输入长度})$ ）	低（最坏指数级）

对比维度	手工编码	自动生成
复杂度	较高，易出错	代码量少，快速原型
控制细度	可精细控制	细节较难控制
典型工具	手写 C/C++ 代码	Lex, Flex, JLex
使用场景	GCC, LLVM	原型/小语言

重点考点清单

- Q1 给定正则表达式，使用 Thompson 算法构造等价 NFA。
- Q2 给定 NFA，使用子集构造算法（含 ε -闭包计算）转换为等价 DFA。
- Q3 给定 DFA，使用 Hopcroft 算法最小化（等价状态划分）。
- Q4 给定语言描述，写出对应正则表达式；或给定正则表达式，描述其对应的正规集。
- Q5 正规集、正规式、有限自动机三者之间的等价性证明（思路）。
- Q6 DFA 的转移表驱动实现方法（尤其最长匹配策略）。
- Q7 标识符和关键字的识别方法（切分转移图 vs 关键字表算法）。

Q8 词法分析器在编译器前端的定位，记号（Token）的数据结构设计。

Q9 正则表达式的性质：交换律、结合律、分配律、单位元。

Q10 语法糖的等价展开： $[a-z]$, e^+ , $e?$, $e\{m,n\}$ 。

重要等价关系

正则表达式 (RE) $\Leftrightarrow$ NFA $\Leftrightarrow$ DFA $\Leftrightarrow$ 最小 DFA $\Leftrightarrow$ 正规文法 $\Leftrightarrow$ 正规集